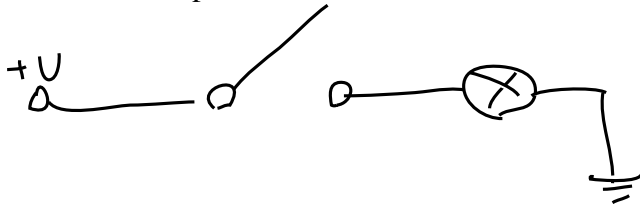


Inbyggda system.

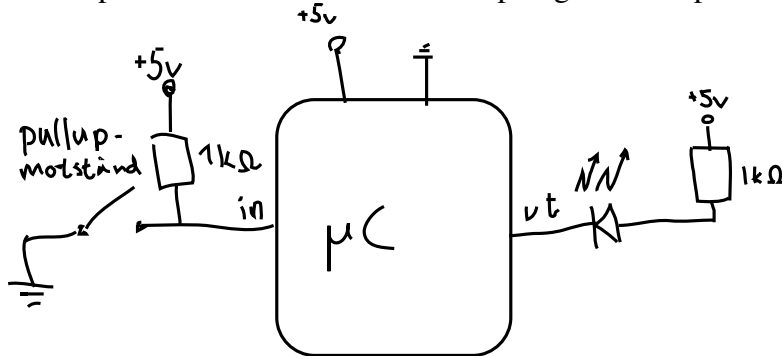
Föreläsning 1
060323

Digital i/o.

Enklaste exemplet.



Hur implementeras denna enkla tillämpning i en enkapseldator?



Digital 1 (hög) då knappen *inte* trycks ned.

Dioden lyser då utgången blir 0 (låg).

På detta sätt är knappar och lysdioder anslutna på STK500.

Vad måste finnas i en enkapseldator för att implementera detta?

- i/o-portar (digitala in- och utgångar)
- CPU/processor
- Minne
 - programminne – permanent, behåller innehållet utan matningsspänning.
 - dataminne – läsa/skriva lika snabbt.

Detta finns i alla mikrodatorer.

Minnen.

Programminne. (non-volatile)

Innehållet måste bevaras utan strömförsörjning.

- **ROM – Read only memory**
Tillverkas med ett visst innehåll som sedan ej kan ändras. Billigt i mycket stora volymer.

- **EPROM – Erasable PROM**
Kan programmeras och raderas flera gånger. Radering sker med hjälp av UV-ljus genom ett fönster på kapseln ungefär tjugo minuter.
- **OTP – One-time Programmable**
EPROM utan fönster.
- **EEPROM – Electrical EPROM**
Kan programmeras och raderas elektriskt. Kan ändra enskilda bytes, vilket är en fördel. Kan ändras i mikrodatorn under körning.
- **FLASH**
Billigare variant av EEPROM där minnescellerna hanteras i större block. Mycket vanligt i enkapseldatorer.

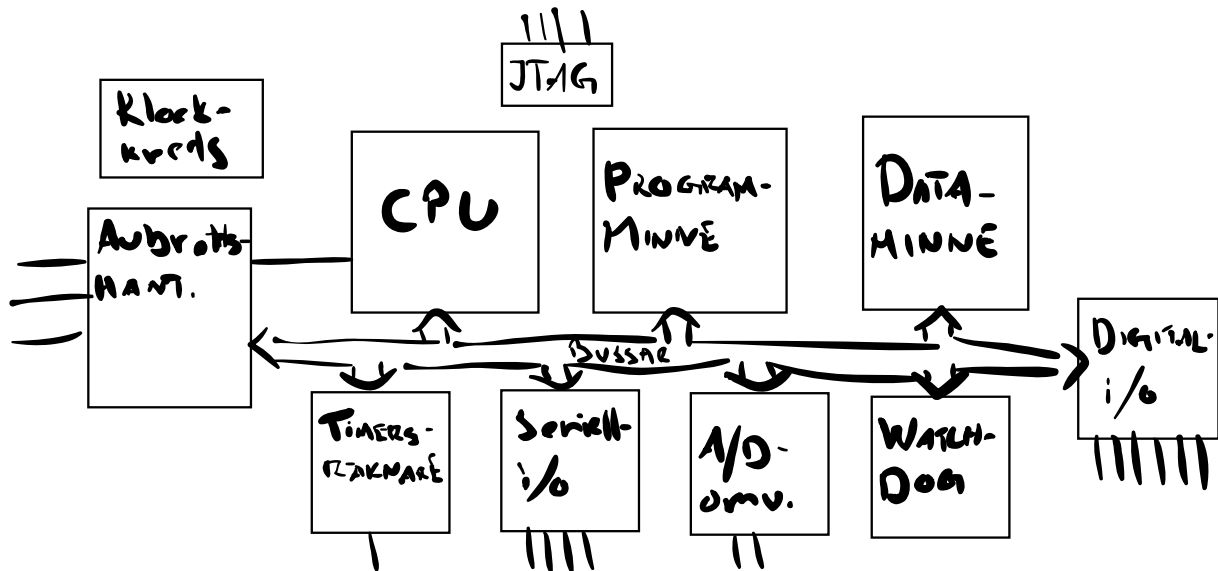
Dataminne.

- **SRAM – static RAM**
Finns i alla enkapseldatorer. (Samma typ som cacheminnet i PC.)
(Möjlighet kan finnas att ansluta DRAM till större datorer.)
- **EEPROM**
Kan också finnas utöver SRAM. Tar längre tid att skriva än läsa. Håller för ca 100 000 skrivningar. Behåller informationen utan ström.

Inbyggda system

Föreläsning 2
060324

Digital i/o (forts)



Watchdog:

Speciell timer som kan användas för att starta om datorn då programmet hängt sig.

Timer/räknare:

”Vanlig” binärräknare som kan räkna händelser, mäta tid eller dylikt.

Digital i/o:

Ofta organiserade i portar om 8 pinnar, vars logiska nivå kan läsas eller skrivas från programmet via minnesceller (register).

Seriell i/o:

Kommunikationskrets för att prata med andra datorer/kretsar.

JTAG:

Kan användas för felsökning och programmering.

A/D-omvandlare:

Omvandlar en analog signal till digital.

Avbrottshanterare:

Genererar avbrott hos processorn, t ex för att externa enheter signalerar.

Bussar:

Beror på arkitektur och ordlängd på CPU, 8 eller 16 bitars ordlängd vanligast i enkapseldatorer (32 eller 64-bitar på PC.)

Några begrepp inom arkitektur:

- von Neumann (CISC)
- Harvard (RISC)

von Neumann:

Alla typer av minne och övriga enheter delar på samma adress- och databuss.

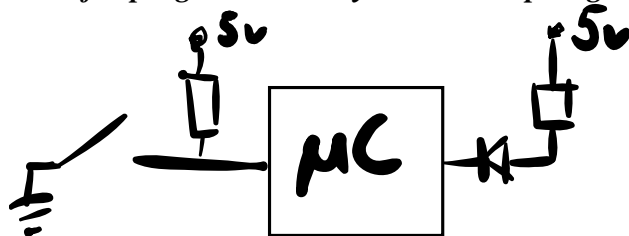
Harvard:

Flera parallella adress- och databussar. Vanligast att programminnet har en buss och resterande delar på en buss. Program- och dataminne kan accessas parallellt.

På labben använder vi Atmel AVR ATmega16 med:

- 8-bitars RISC-processor med Harvard arkitektur.
- 16 kB FLASH-minne för program.
- 1 kB SRAM.
- 512 byte EEPROM
- 3 räknare/timers
- A/D-omvandlare (10 bitars, 8 ingångar, 15 ksample/s)
- Inbyggd oscillator
- JTAG-gränssnitt
- Seriella gränssnitt:
 - UART (RS-232)
 - SPI
 - I²C

Skiss för programmet till lysdiodstillämpningen:



```
//Definiera en pinne i en i/o-port som ingång och en som utgång.
while(TRUE)
{
    //Läs logiska värdet på ingången, skriv ut värdet till utgången.
}
```

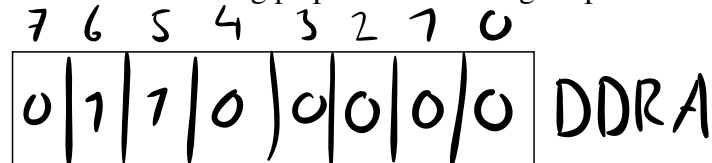
De digitala in- och utgångarna är organiserade i form av portar med 8-bitar i varje. mega16 har 4 portar A-D. Utgå från att vi använder mega16.

Till varje port (A-D) finns 3 minnesadresser (register) kopplade. Dessa adresser har symboliska namn i databladet:

DDRx, PORTx och PINx

DDRx: (Data Direction Register)

Bestämmer riktning på pinnarna som ingår i porten.

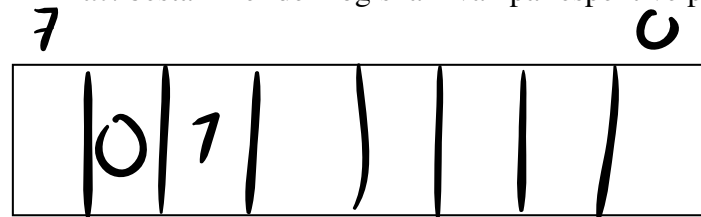


Genom att skriva en 1:a till en viss bit i DDRx sätter motsvarande pinne i porten *till utgång*.
0:a ger ingång.

PORTx:

Används för olika saker beroende på portens riktning.

ut: bestämmer den logiska nivån på respektive pinne.



in: en 1a aktiverar internt pullupmotstånd. Pullupmotståndet ser till att ingången har en meningsfull logisk nivå.

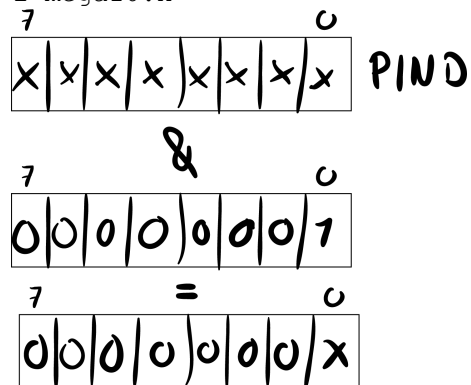
PINx:

Läser den logiska nivån på pinnarna. Fungerar för både in- och utgångar.

Alt1.

```
#include <mega16.h>          <-- definition av chip, ersätts med  
                               innehållet i mega16.h
```

```
void main(void) {  
    // port D görs till ingång  
    DDRD = 0;  
    // port B görs till utgång  
    DDRB = 0xFF; ( = 0b11111111)  
  
    while(1) {  
        if(PIND & 1)  
            PORTB = 1;  
        else  
            PORTB = 0;  
    }  
}
```



Detta är ANSI-C, fungerar på alla kompilatorer. Kompilatorn vi använder, CodeVisionAVR, innehåller många tillägg som kan göra programmen kortare och mer läsbara:

I CodeVision:

```
#include <mega16.h>  
  
void main(void)  
{  
    DDRD.0 = 0;  
    DDRD.0 = 1;  
    while(1) {  
        PORTB.0 = PIND.0;  
    }  
}
```

I ANSI-C

```
DDRD = DDRD & 0xFE;  
DDRB = DDRD | 0x01;
```

Inbyggda system

Föreläsning 3
060327

Programmeringsspråk

Enkapseldatorns assemblerspråk

- Fördelar med assembler
 - Effektivt: prestanda, programminne, minnesanvändning → billigare produkt
 - Kontroll på vad som händer.
 - Full kontroll över hur tid ett visst kodavsnitt tar.
 - Ger möjlighet att utnyttja alla finesser/funktioner som finns i chippet.
- Nackdelar
 - Hårdvaruberoende
 - Oöverskådligt
 - Stor kodmassa.
 - Ligger långt ifrån vårt sätt att tänka.

Om nackdelarna väger över kan ett lämpligt högnivåspråk väljas istället:

- C (C++)
- Ada
- Java – J2ME
- Forth

Vanligast idag är *C*.

Positivt:

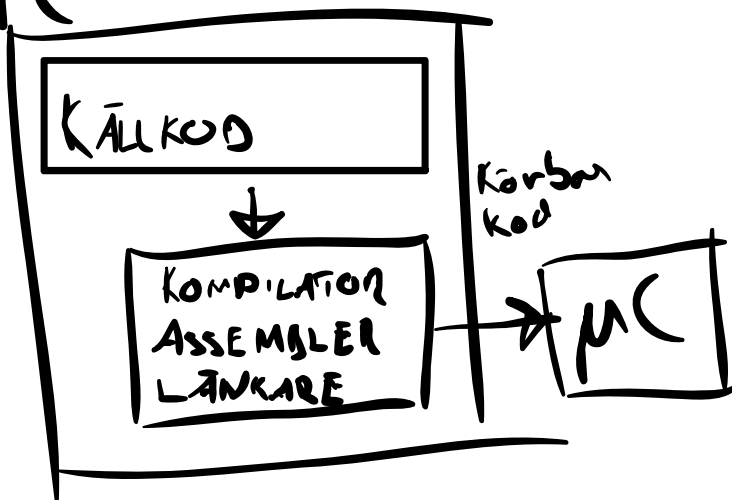
- Mer portabelt än assembler.
- Effektivt (snabbt och lite kod).

Negativt:

- Saknar felkontroll som i många andra språk. Detta är dock en av anledningarna till att koden blir liten och snabb.

Programutveckling

PC



Hur felsöker man?

Blinka med LED:ar, skriva ut text på LCD, skicka och ta emot text via terminal.

Alla extra rader med kod för att kunna felsöka leder till många överflödiga programrader. För att administrera detta kan man använda pre-processor-direktiv.

`#define` – definierar ett makro, t ex

```
#define LED PORTB (LED.3 = 1; => PORTB.3 = 1).
```

Alla förekomster av LED byts mot PORTB.

`#define DEBUG`

`#ifdef` – testar om ett makro är definierat, t ex

```
#ifdef DEBUG
```

```
// Skriv ut värdet av en variabel på LCD
```

```
#endif
```

Detta tar inget extra minne i mikrodatorn, när `#define DEBUG` kommenteras bort försvinner all kod mellan `#ifdef DEBUG` och `#endif`.

Verktyg för felsökning

Simulator

Program på PC:n som simulerar enkapseldatorn. Gör det möjligt att följa programkörningen rad för rad, titta/ändra variabler och register.

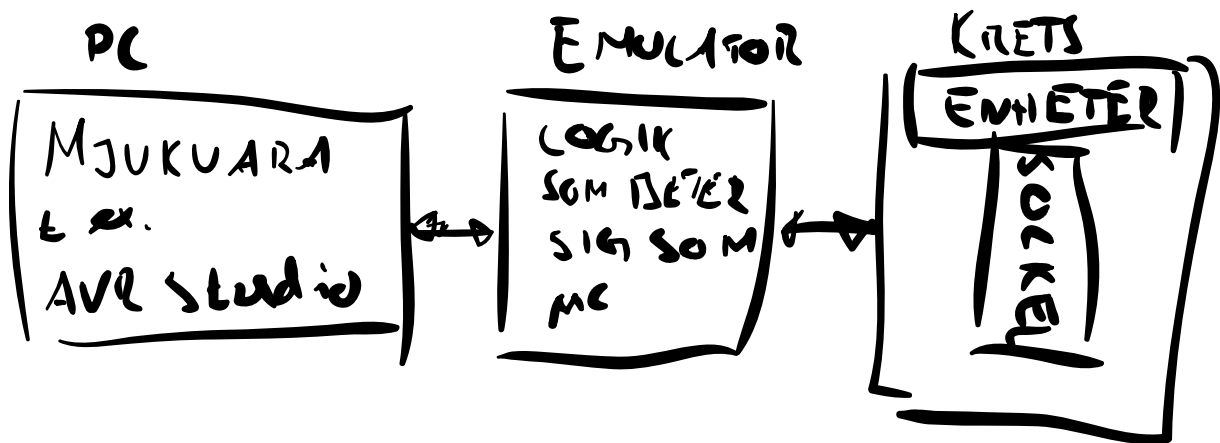
Långsammare än hårdvara, ingen kontakt med omvärlden.

AVR Studio har en inbyggd simulator för hela AVR-familjen.

Oscilloskop

Kan användas för att mäta exekveringstider för funktioner. T ex sätt en pinne på chippet till 0 eller 1 och mät denna.

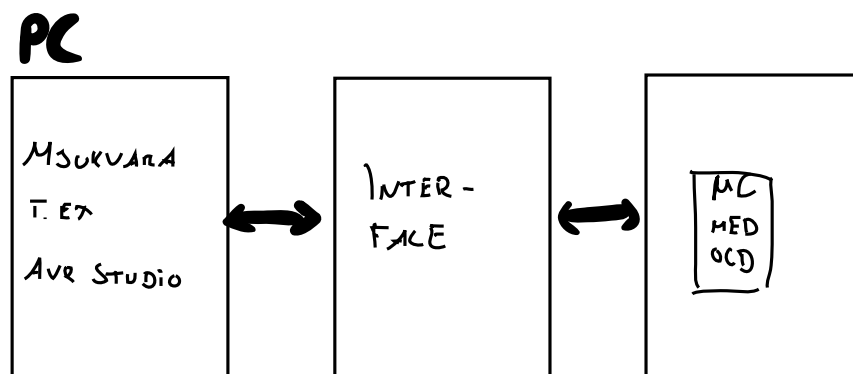
Emulatorer



Äkta emulator:

Annan hårdvara emulerar enkapseldatorn. Tidigare händelser kan analyseras vid stopp. Nackdelen är att alla interna enheter inte emuleras med exakt timing Oftast dyra.

OCD – On Chip Debugging



Finns i många enkapseldatorer. Inbyggd funktion i enkapseldatorn för debugging och felsökning. Stoppar mikrodatorns exekvering vid bestämda händelser eller från PC. Alla register och minne laddas över till PCn. Nackdelen är att det inte går att se vad som hänt när processorn kördes.

I labbet har vi **JTAG ICE mhII**. Några möjligheter:

- Förutom att stega, starta och stoppa körningen manuellt finns
 - Breakpoints
 - Data: På läsa, skriva eller läsa/skriva till vissa minnesadresser eller i/o-register. Fungerar ej på processorns interna register, t ex R0-R31.
 - Program: Kan sättas att stanna på valfria platser i programmet.

Inbyggda system

Föreläsning 4
060328

Digital i/o (forts)

Logiska och bitvisa operatorer

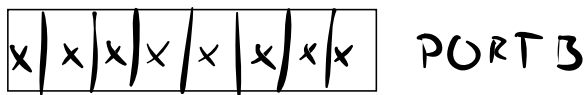
Kom ihåg:

PORTB.0 = 1;

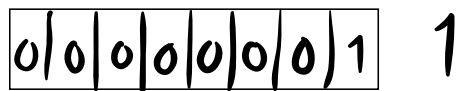
Fungerar bara i CodeVision.

I ANSI-C gäller:

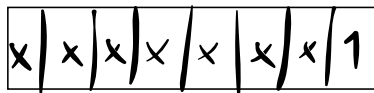
PORTB = PORTB|1;



OR (bitvis)



=



Talrepresentation:

Exempel

Decimalt → 1, 10, 24

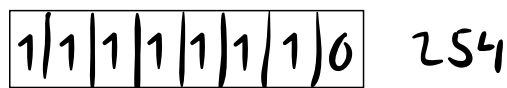
Binärt → 0b00000100 = 4

Hexadecimalt → 0x04 = 4 (0x → hexadecimalt)

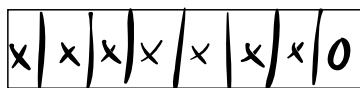
För att sätta en bit till 0 används & (bitvis AND)



AND (bitvis)



=



Följande bitvisa operatorer finns:

&	AND
	OR
^	XOR
~	NOT

Ex:

char a = 0b1111000, ~a
blir resultatet 0b00001111;

Dessa skiljer sig från de logiska operatorerna. Alla tal i C har ett logiskt värde; 0 är falskt, logiskt 0. Alla andra tal är sant, logiskt 1.

T ex. -10, 0.01, 100 är sant.

Resultat från logiska operationer är alltid 0 eller 1.

Logiska operatorer:

&&	AND
	OR
!	NOT

Ex:

char a = 0b0000010;
char b = 0b0000011;

a&b => 0b00000010 = 2

a&&b => 0b00000001 = 1

I C skiljer man på tilldelning och jämförelse. "Alla satser är uttryck". Vanligt fel!

while(a==3)

Sant om a har värdet 3, annars falskt.

while(a=3)

Tilldelar a värdet 3. Uttrycket har värdet 3, alltså alltid sant.

Första exemplet var ett program som tände en lysdiod då en brytare trycktes ned.

Tidsanalys av program.

C-kod	Assembler	
#include <megal6.h> void main(void) { DDRD.0 = 0; PORTD.0 = 0; DDRB.0 = 1; while(1) { PORTB.0 = PIND.0 } }	CBI 0x11,0 //Clear Bit 11 vid plats 0 CBI 0x12,0 SBI 0x17,0 //Set Bit _0x2: CLT SBIC 0x10,0 SET IN R26,0X14 BLD R26,0 OUT 0X18,R26 RJMP _0X2	2c 2c 2c 1c 1-2c 1c 1c 1c 1c 2c } 1.5µs } ≥ 2.25µs

```
while(1)
{
    PORTB.0 = PIND.0
    ~ 8µs
} } ≥ 2.25µs
```

Responstid på max 10µs ⇒ ~8µs

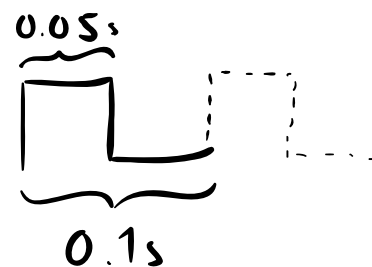
Exempel 2.

Modifierar exemplet så att lampan blinkar med 10 Hz då strömbrytaren är sluten. Vi kan behålla samma hårdvara som i förra exemplet.

```
#include <mega16.h>

void blink(char antalgr);
void main(void) {
    // initiera hårdvaran (portar enl. ex 1)
    while(1) {
        if (PIND.0 == 0)
        {
            blink(1); //Blinka lysdioden en gång
        }
    }
}
```

```
void blink(char antalgr)
{
    char i;
    int j;
    for(i = 0; i < antalgr; i++)
    {
        PORTB.0 = 0; //Tänd lysdioden
        for(j = 0, j < 25000, j++);
        PORTB.0 = 1;
        for(j = 0, j < 25000, j++);
    }
}
```



Välj så att loopen tar 0.05s.

Heltalsvariabler i C

Det finns tre olika "storlekar".

char ≤ int ≤ long int

I CodeVision 8 bit 16 bit 32 bit

Observera att "char" är en helt vanlig heltalstyp, den har inte med tecken att göra. Tänk på att använda så små datatyper som möjligt. Till varje datatyp kan vi lägga modifierarna `unsigned` resp `signed`. Om vi inte skriver något tolkas detta som `signed`. Standard i projektinställningen i CodeVision är dock att "char" tolkas som `unsigned`.

Talområden

	signed	unsigned
char	-128 – +127	0 – 255
int	-32768 – +32767	0 – 65535
long int	-2·10 ⁹ – +2·10 ⁹	0 – ~ 4·10 ⁹

Alt 1. $i++$;
Alt 2. $i = i + 1$;
Alt 3. $i + = 1$;

$+=$ fungerar med alla tal, t ex $1 + = 5$

$|=$ Bitvis OR t ex `PORTB.1 = 1`; samma som `PORTB = PORTB | 1`;

Anm.

For-loopar i C

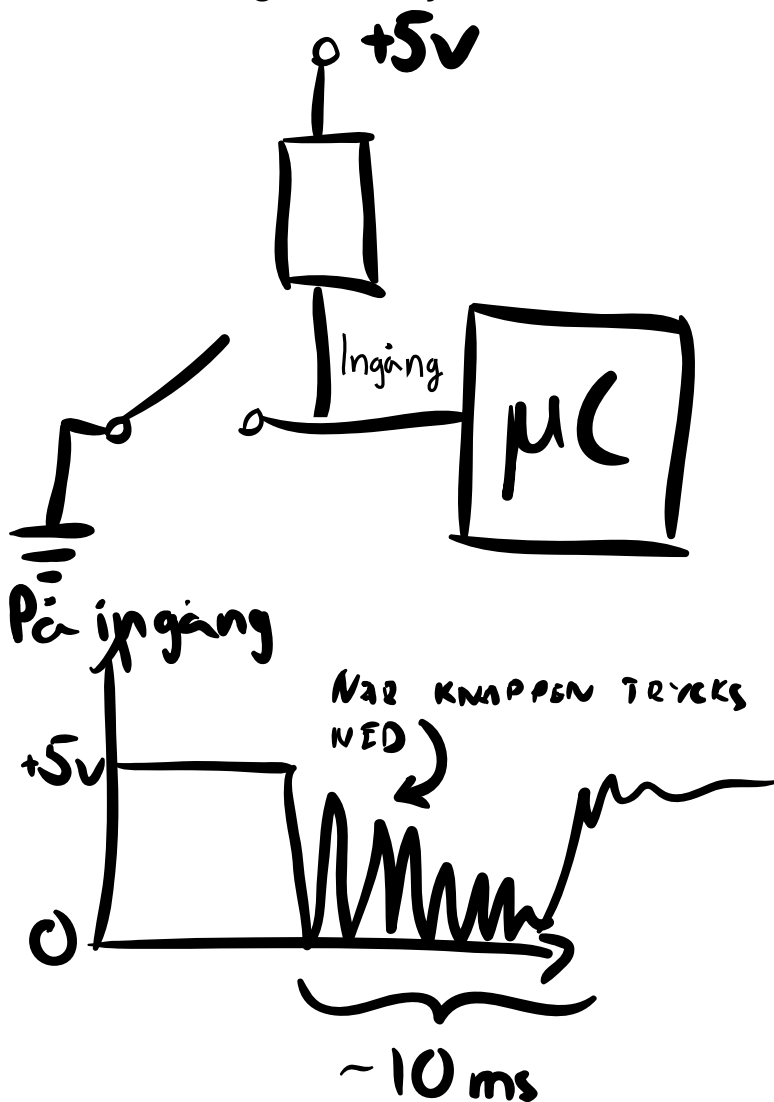
for(, ,)

körs efter varje
varv

körs en gång före
första loopvarvet

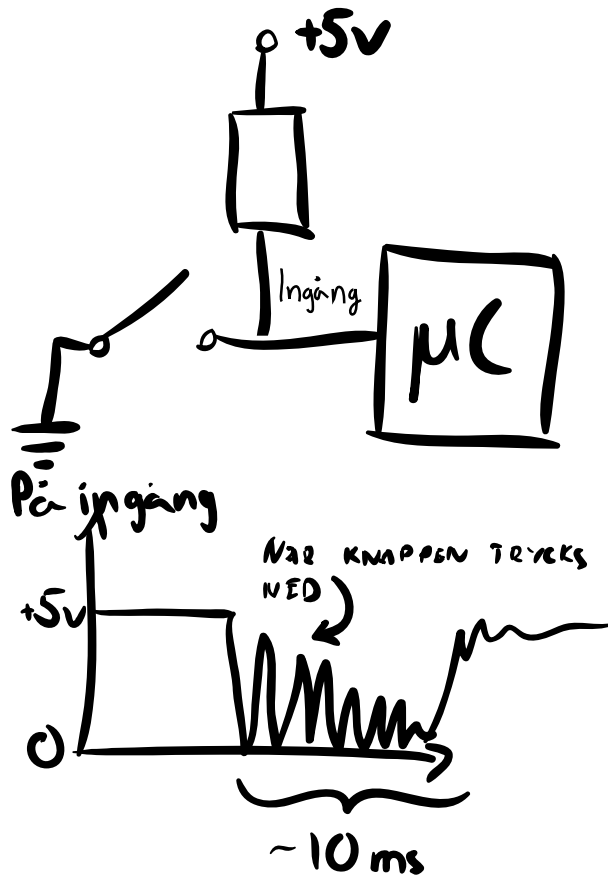
villkor som
beräknas varje
loopvarv, SANT
 $\Rightarrow$ varvet körs

Kommentar ang. strömbrytare



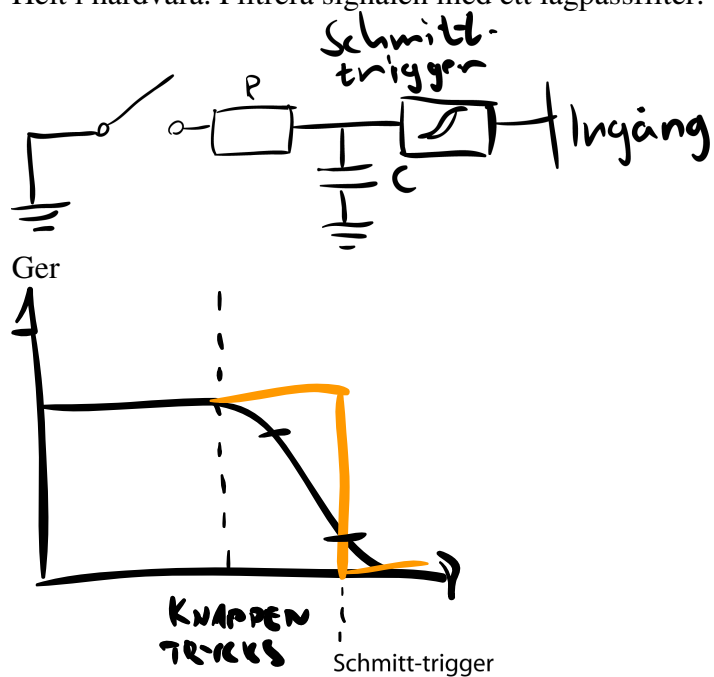
Inbyggda system

Föreläsning 5
060529



Lösning 1.

Helt i hårdvara. Filtrera signalen med ett lågpasfilter.



Lösning 2.

Helt i mjukvara:

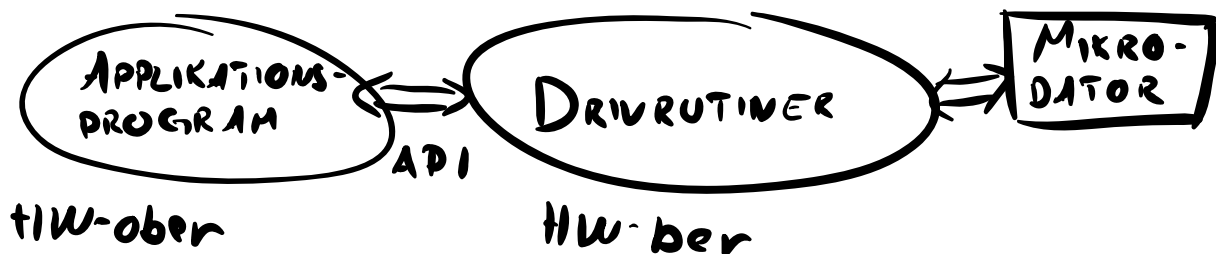
lägg in en avsiktlig fördröjning i programmet.

- Slösar CPU-tid
- Svårare att implementera partiella uppgifter

Lösning 3.

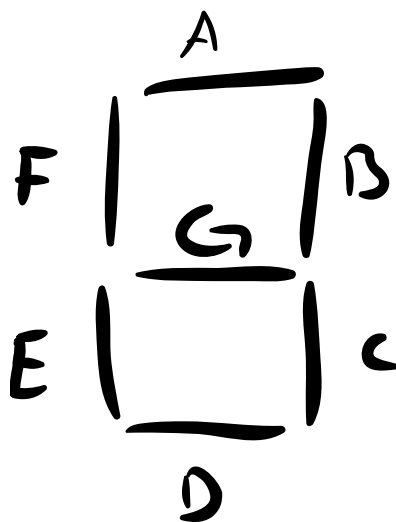
Mixad hårdvaru- och mjukvarulösning. Används någon form av tidtagning för fördröjningen. I de flesta mikrodatorer finns minst en timer som kan användas för tidtagningen.

Tidigare exempel har haft hårdvaruberoende kod utspridd i hela programmet. Mycket arbete om hårdvaran ändras i framtiden. Bättre lösning är att samla hårdvaruberoende kod i ett litet antal funktioner – drivrutiner.

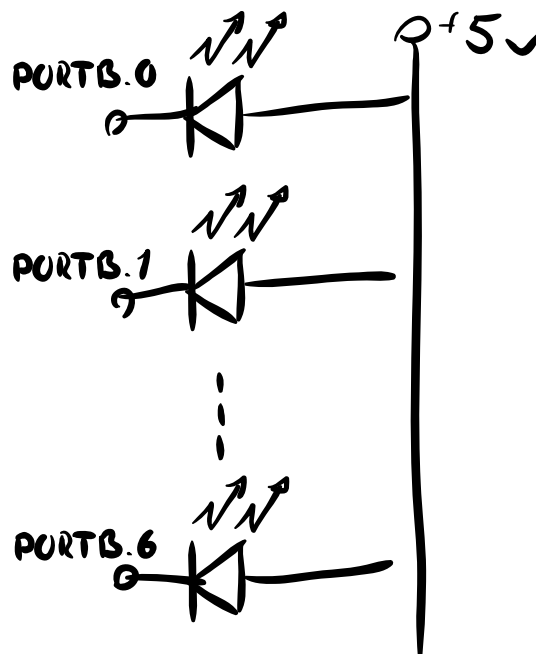


Exempel 3.

Anslutning av en 7-segmentsdisplay.

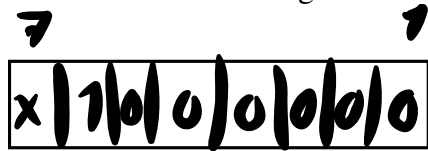


Består av 7 lysdioder.



- Displayen kan anslutas direkt till en port på mikrodatorn, och ett segment kan tändas genom att lägga ut 0 på ansluten pinne.

PORTB är ett 8-bits register.



0:  0b01000000 = 0x40

1:  0b01111001 = 0x79

Dessa lagras enklast i en tabell:

Alt 1.

```
char siffra[10];  
siffra[0] = 0x40;  
siffra[1] = 0x79;
```

Lagras i SRAM.

Alt 2.<

```
char siffra[] = {0x40, 0x79, ...};
```

Alt 3.

```
const char siffra[] = {0x40, 0x70, ...};
```

const (alt. flash) talar om för kompilatorn att innehållet inte kommer att ändras. Arrayen behöver därför inte lagras i SRAM utan kan ligga kvar i flash-minnet.

ASCII

Standard för representation av bland annat bokstäver och siffror.

Exempel.

'A' = 65

'1' = 49

'!' = 33

Exempel i C.

```
char c;  
c = 'A';  
if (c == 'A')  
{  
    ...  
}
```

```
char c;  
c = 'A';  
if (c == 65)  
{  
    ...  
}
```

```
#define _ALTERNATE_PUTCHAR_ //Byt ut putchar  
const char siffra[] = {0x40, ...}; //Måste vara global  
char putchar(char c) //c är en ASCII-kod  
{  
    if (c < '0' || c > '9')  
    {  
        return 1;  
    }  
    else  
    {  
        PORTB = siffra[c - '0'];  
        return 0;  
    }  
}
```

Tack vare att vår funktion heter just `putchar()` kommer samtliga utskriftsfunktioner i standardbiblioteket att använda vår funktion.

Avbrottshantering

I tidigare exempel låg programmet i en loop och måste med jämna mellanrum kontrollera om någon tryckt på knappen. För att inte missa en tryckning måste vi kolla ofta, samtidigt sker tryckningar relativt sällan → slöseri med resurser.

Bättre är att låta ingången ”tala om” för resursen att någon tryckt på knappen.

När en avbrottssignal anländer till en av processorns avbrottsingångar sker följande:

1. Spara undan det nödvändigaste för att kunna fortsätta (text position i programmet när avbrottet kom) programmet efter avbrottsfunktionen är klar. Absolut minimum är att spara undan adressen till aktuell programinstruktion. (Detta gör processorn automatiskt).
2. Hoppa till (anropa) den funktion som är knuten till den aktuella avbrottssignalen.
3. När avbrottsfunktionen är slutförd, återställs den sparade informationen och exekveringen fortsätter där den avbröts.

OBS! När ett avbrott exekveras kan inte andra avbrott avbryta, däremot kan man manuellt aktivera avbrott för att tillåta så kallade nästlade avbrott. Kan vara farligt om man inte noga kontrollerar att avbrotten avbryter sig själva.

Exempel 4.

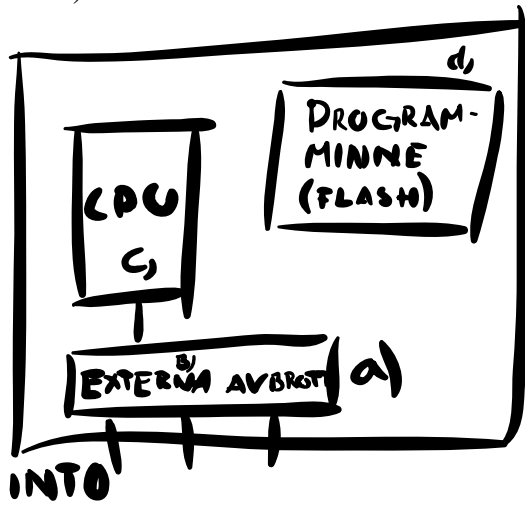
Skriv om den tidigare lysdiodsapplikationen med ett avbrott från knappen. Vi kan använda pinnarna märkta:

INT 0, INT 1 eller INT 2 på kapseln.

(Fysiskt samma anslutningar som bit 2, 3 i PORTD och bit 2 i PORTB)

För att avbrottet ska fungera måste följande utföras:

- Aktivera och konfigurera INT0 i hårdvaran.
- Aktivera sändning av avbrottssignalen till processorn (CPU).
- Aktivera mottagning av avbrottssignalen i processorn.
- Skriva en avbrottsfunktion och koppla denna till INT0-avbrottet.



Inbyggda system

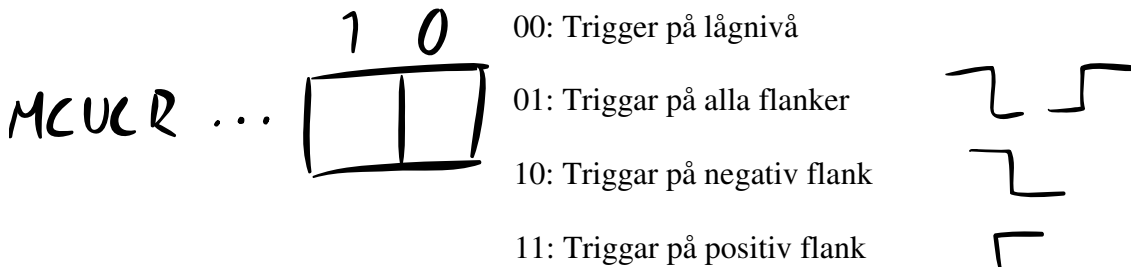
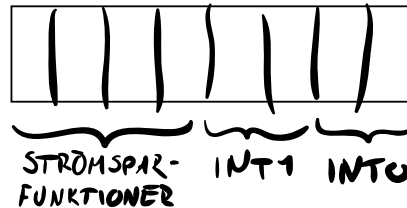
Föreläsning 6
060529

Lysdiodsapplikationen med avbrott

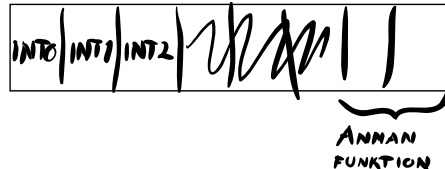
- Konfigurera iNT0 (Delas med pinne2, PORTD)
- Aktivera sändning av avbrott INT0.
- Aktivera global avbrottsshantering i CPU.
- Skriv avbrottsfunktionen.

3 register är associerade med INT0.

- MCUCR



- GICR



Styr vilka avbrott som är aktiverade.

GIFR



Ställs till 1 då ett avbrott genererats. Nollställs genom att skriva en 1:a till biten.

- Avbrott aktiveras globalt i processorn genom instruktionen `sei`. I C-kod:
`#asm("sei")`. För att deaktivera avbrott globalt används `cli`, `#asm("cli")`.

C-koden för applikationen:

```
#include <mega16.h>
void main(void) {
    DDRB.0 = 1;    // Utgång
    PORTB.0 = 1;   // LED släckt
    DDRD.2 = 0;    // INT0 är ingång
    PORTD.2 = 0;   // Deaktivera interna pullup-motstånd
    MCUCR = 0x01;  // Triggas INT0 på alla flanker
    GICR |= 1<<6; // Sätt bit 6 till 1
```

// bitvis 1 eller 0-ställning fungerar på i/o-register med adress lägre än 0x1F (de 32 första) //

```

    #asm("sei")    // Sätter bit 7 i processorns statusregister SREG
    while(1)
    {

    }
}
//Avbrottsfunktion i CodeVision
interrupt [EXT_INT0] void avbrott(void)
{
    // Eventuell tidsfördröjning för att undvika kontaktstuds. Rensa GIFR.
    if (PIND.2 == 0)
        PORTB.0 = 0;
    else
        PORTB.0 = 1;
}

```

deklarerad i mega16.h

Avbrott deaktiveras automatiskt när processorn börjar exekvera en avbrottsfunktion och aktiverar dem igen när avbrottsfunktionen är färdig.

Ang. `cli`: Kan vara bra vid tidskritiska avsnitt i koden. `sei`

Strängar och LCD

Man måste skilja på strängar i SRAM och FLASH.

FLASH: `const char str1 = "Hej";`

SRAM: `char str2 = "Hej2";`

Strängar i FLASH: `strcpy(s, str1);`

Strängar i SRAM: `strcpy(s, str2);`

16 TECKEN
16 TECKEN

Initiera LCD: `lcd_init(16);` //Antalet tecken per rad

Skriva en sträng: `lcd_puts(str2);` //SRAM
 `lcd_putsf(str1);` //FLASH

Rensa LCD: `lcd_clear();`

Gå till (x,y): `lcd_gotoxy(x,y);`

Wizard: Välj PORT till LCDn.

Räknare och timers

- Samma krets
- Binnärräknare
- Vanligen 8 eller 16 bitar
- Räknarens värde kan läsas och skrivas från programmet

Timer

Pulsarna har konstant frekvens (vanligen från systemklockan). Används för tidtagning.

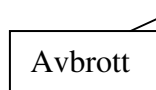
Räknare

Pulsarna kommer från okänd källa (ingång på chippet). Används för pulsräkning.

Alla timerkretsar kan generera avbrott när räknaren slår runt.

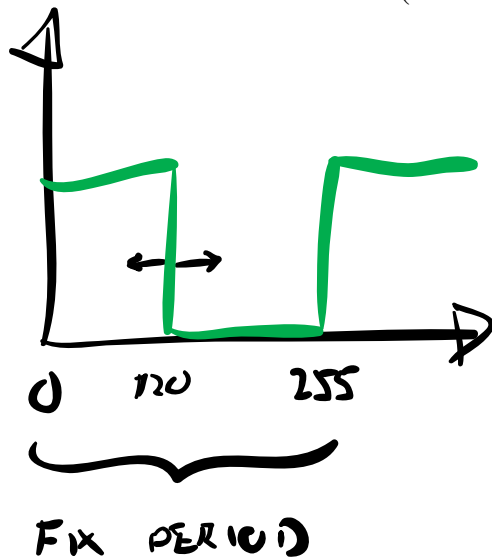
Timer0 i mega16.

8 bitar: 0, 1, ..., 255, 0



Ytterligare funktioner som kan finnas i mer avancerade timerkretsar.

- **Output Compare:** Avbrott genereras vid valfritt värde på räknaren.
- **Input Capture:** När en signal (t.ex. negativ flank) anländer till en speciell ingång sparas räknarens värde i ett särskilt register. Ett avbrott genereras.
- Pulsbreddsmodulation (PWM – Pulse Width Modulation)

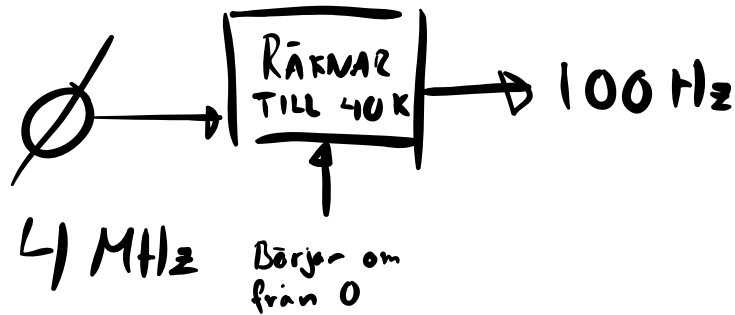


Inbyggda system

Föreläsning 7
060529

Exempel på timeranvändning.

En viss programfunktion kör 100ggr/sek (10ms intervall). Detta kan realiseras med en timerkrets som genererar avbrott 100ggr/sek.



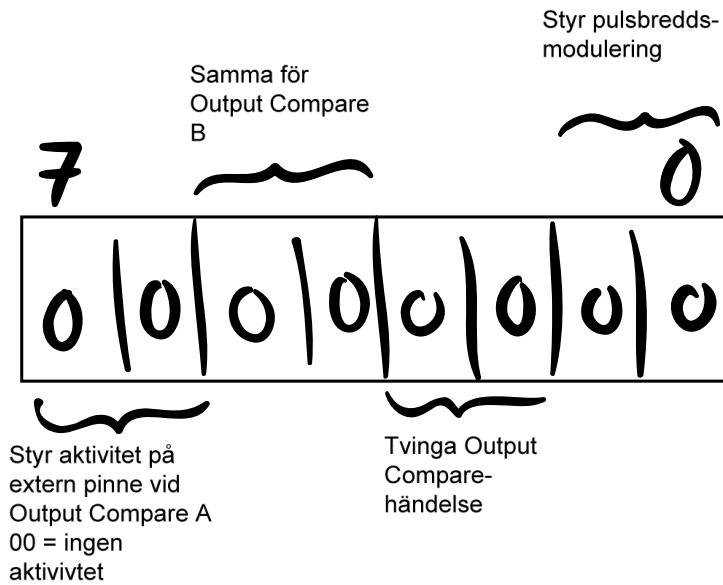
Detta kräver 16-bitars timer. Timer1 i mega16 är 16-bitars.

Register i Timer1

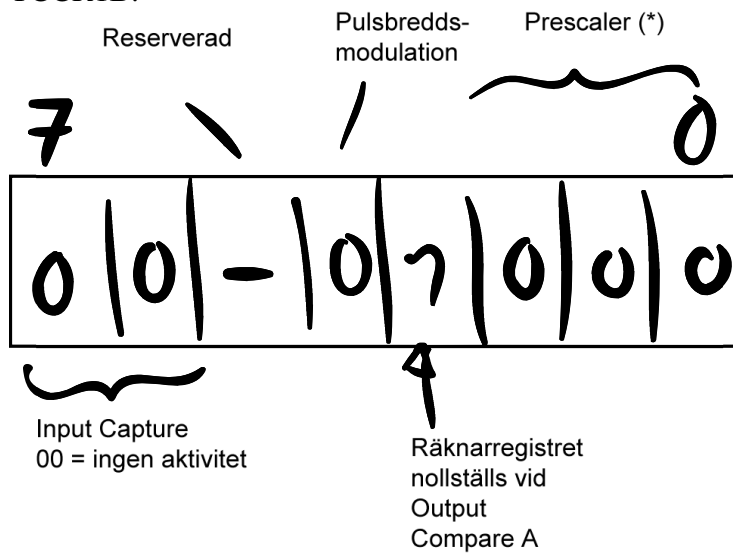
- Räknaarregistren (själva räknarens tillstånd lagras i)
TCNT1 – 16 bitarsregister
TCNT1H – Hög byte
TCNT1L – Låg byte
Timern ser det som ”ett” 16-bitarsregister, 8-bitsprocessorn som 2 st 8-bitars.
- Output Compare: OCR1A – (H/L) (Generera avbrott vid valfritt räknarvärde)
OCR1B – (H/L)
- Input Capture: ICRI (H/L) (För att t.ex. mäta frekvens)
- Kontrollregister: TCCR1A/TCCR1B
- Avbrottskontroll: TIMSK
- Statusflaggor: T1FR

Konfigurering

TCCR1A:



TCCR1B:



(*)

Timer	000 – Timer avstängd 001 – Klockas av systemklockan (t.ex. 4MHz) 010 – Klockas av systemklockan/8 (500 kHz) ... 101 – Klockas av systemklockan/1025 (~4kHz)
-------	---

Räknare	110 – Extern klocksignal neg. flank 111 – Extern klocksignal pos. flank
---------	--

Exempel 5. Blink med timers.

10 Hz blinkningar. Kollar både knapp och sköter blinkningar med en timer.

```
#include mega16.h
void main(void)
{
    //Konfigurera i/o
    PORTB = 0xFF;           //Släcka LED på PORTB (utgång)
    DDRB = 0xFF;
    PORTB.3 = 1;           //Pull-up
    DDRD.3 = 0;            //Ingång
    //Konfigurera Timer1
    TCCR1A = 0x00;
    TCCR1B = 0b00001010;    //Börja om vid Output Compare,
                           //systemklockan/8
    OCR1A = 25000;         //Avbrott 20 ggr/sek
    TIMSK = 0x10;          //Aktivera Output Compare A avbrott
    #asm("sei")
    while(1)
    {
    }
}

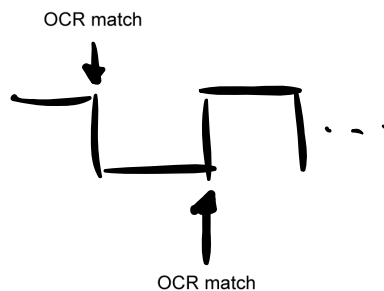
interrupt [TIM1_COMPA] void avbrott_timer1(void)
{
    if(PIND.3 == 0)
        PORTB.5 = ~PORTB.5;
    else
        PORTB.5 = 1;
}
```

Alt. Använd Output Compare med vågformsgenerator.

- Använd timer0
- Varje Output Compare byter nivå på pinne OCR1 (delas med PORTB.3)
- Lägsta klockfrekvens systemklocka/1024
- Output Compare vid 196 => 20 OC/sek.

Program:

```
//i/o samma som ovan
//timer0 konfigureras
TCCR0 = 0x18;           //Konfigurera registertimer0
TCNT0 = 0x00;           //Nollställer timer
OCRO = 196;             //Output Compare vid 196
while(1)
{
    if(PIND.3 == 0)
        TCCR0 = 0x18;
    else
        TCCR = 0x00; //Timer av
```



Deklaration av globala variabler för användning i avbrottsfunktionen och andra funktioner

Exempel

```
char a = 1; //Global
interrupt [EXT_INT0] void fun(void)
{
    //a manipuleras
}
void main(void)
{
    //a läses och manipuleras
}
```

För att undvika problem p.g.a. t.ex. kompilatorns optimering så deklarerar man dessa variabler som volatile (flyktig).

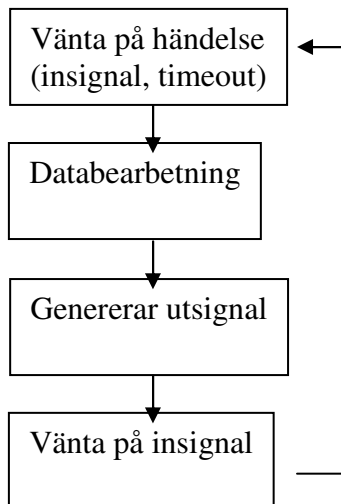
```
volatile char a = 1; //Global
```


Inbyggda system

Föreläsning 8
060529

Multitasking utan operativsystem

Deluppgifter (processer) i ett inbyggt system kan normalt beskrivas som:



- För att processerna ska kunna köras parallellt kan koden inte skrivas sekventiellt

Ex:

```
while()  
{  
    //Vänta på insignal  
    //Gör beräkning  
    ...  
}
```

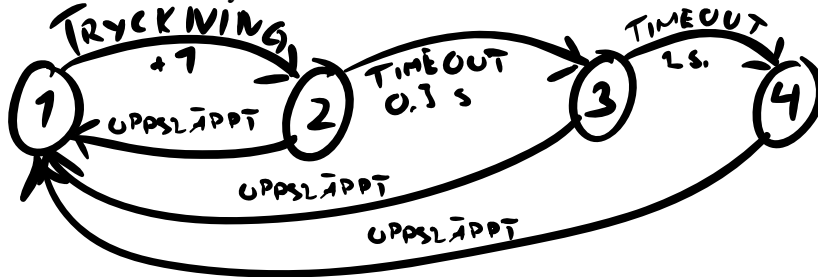
Lösning:

Dela upp koden i små bitar och minimera informationen som behövs vid processbyte
=> Gör små tillståndsmaskiner

Exempel.

Använd en tillståndsmaskin för att få repeteringsfunktion på en tryckknapp.

- När knappen trycks ned – öka en variabel med ett.
- Om nedtryckt > 0.3 sek – öka med 100/sek
- Om nedtryckt > 2 sek – öka med 1000/sek



3: +100/s , 4: +1000/s

Hur ska man implementera detta?

Händelse 2-3, 3-4 kräver tidmätning => analys av tidsupplösning. Tryckningen kräver 10ms upplösning.

- Konfigurera en timer att ge avbrott var 10e ms.
- Avbrottsfunktionen: Kontrollera tillståndet – behöver något utföras?
- Tillståndet lagras som ett tal i en variabel.

Variabeln måste bevaras mellan avbrotten

- Globala variabler gör det
- Static är bättre om vi inte variabeln i andra funktioner

Programexempel

```
unsigned int result; //Global, resultat från process
interrupt [TIM1_...] void avbrotttim1(void)
{
    static char statel = 1; //0 om inte annat anges
    static unsigned int timel = 0;
    timel ++;
    //Tillståndsmaskinen kan beskrivas med en switch-case-sats
    switch(statel)
    {
        case 1:
            if(knapptryckning)
            {
                static1 = 2;
                result ++;
                timel = 0;
            }
            break;
        case 2:
            if(knapp uppsläppt)
                static1 = 1;
            else if(timel == 30) //0.3sek == 300ms = 30*10 ms
                static1 = 3;
            break;
```

```

    case 3:
        result += 1;
        if(knapp uppsläppt)
            static1 = 1;
        else if(time1 == 200)
            static1 = 4;
        break;
    case 4:
        result += 10;
        if(knapp uppsläppt)
            static1 = 1;
        break;
    default:
        static1 = 1;
}
}

```

- Bra uppgifter där vi kan förutsäga när och hur ofta de måste utföras.
- Två typer av händelser som ej är lämpliga i en timerfunktion.
 - 1) Asynkrona händelser: Vet inte när det inträffar, men responstiden måste vara låg (< tidsenhet för timeravbrott)
 - 2) Icke tidskritiska uppgifter
- 1) Asynkrona händelser

Eget avbrott. Tillåts avbryta andra avbrott, d.v.s. lägg in #asm("sei"). Risken är att avbrott avbryter sig själva, inget problem om asynkrona händelsen exekverar snabbt.
- 2) Icke tidskritiska

Placeras lämpligen i huvudloopen i main-funktionen.
Exempel. Skrivning till LCD.

Casting.

```

int sum;
char a, b, c;
a = 200;
b = 65;
c = 3;
sum = a + b;

```

Vad blir sum? 9! Additionen utförs först. a och b är 8-bitarsvariabler => 8-bitsaddition. Om operationen utförs på 2 olika typer gäller följande:

- en är float => float-operation
- En är long => long-operation (32-bitar)
- En är int => int-operation (16-bitar)

Lösning:

```
sum = (int)a + b;
```

(int) kallas casting och betyder att a skall fungera som en int.

sum = 265 enl (3).

Exempel:

```
sum = (int)a + a * c;
```

Multiplikationen utförs först => 8-bits multiplication

```
sum = a + (int)a * c //Rätt
```

Inbyggda system

Föreläsning 9
060529

Seriell kommunikation

- Data som består av mer än en bit skickas efter varandra.
- Sparar på antalet dataledningar.
- Teoretiskt långsammare, gäller dock inte vid höga frekvenser.
- För att kommunikationen ska fungera behöver sändare och mottagare vara överens om när varje bit uppträder, d.v.s. de måste synkroniseras på något sätt.

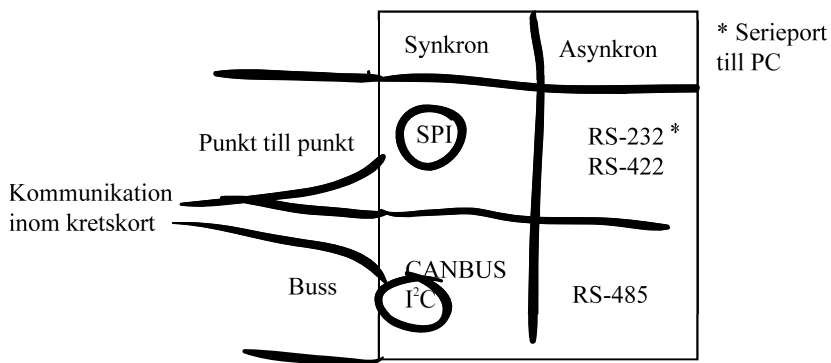
Två huvudtekniker:

- Synkron:
Klockan är gemensam för sändare och mottagare. Den måste överföras på något sätt.
- Asynkron:
Ingen gemensam klocksignal. Extra data behöver skickas

Synkron överföring kan göras snabbare.

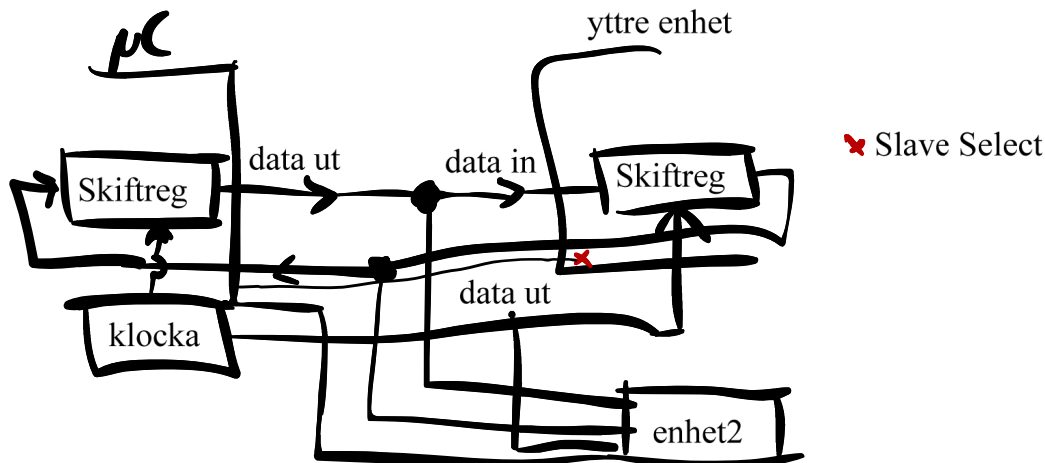
Två andra huvudgrupper:

punkt-till-punkt buss
(en - till en) (många – till många)



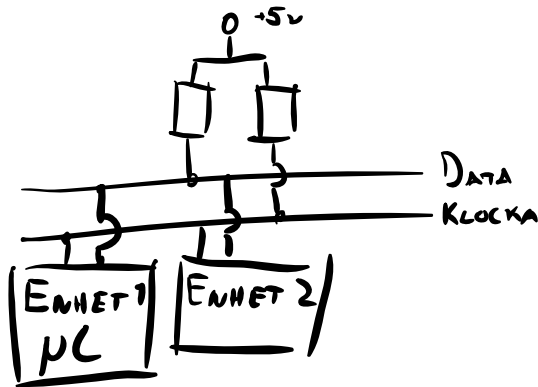
I mega16 finns inbyggda enheter för SPI, I2C och RS-232, RS-485

SPI – Serial Peripheral Interface



- Fullduplex – data kan skickas och tas emot samtidigt
- Bra för dataströmmar – snabb
- Används för t.ex. minnen och kommunikationsenheter
- Kan byggas ut för att hantera flera slavar
 - Anslut en styrledning individuellt till varje slav.
 - Många slavar ger många styrledningar.

I²C - Inter Integrated Circuit bus



- Data- och klockledningar normalt höga via pullup-motstånd.
- En eller flera masterenheter, (vanligen µC)
- Totalt upp till 127 enheter.
- En master startar kommunikation genom att skicka en startsignal (dataledning låg, där efter klocka låg).
- 7-bitar lång adress väljer mastern en enhet att kommunicera med.
- Data skickas genom att lägga ut bit för bit på dataledningen tillsammans med klocksignalen.
- Aktivt låg för 0:a
- Frikopplad för 1:a.
- Realiseras med valfri i/o-port genom att byta riktning på porten.

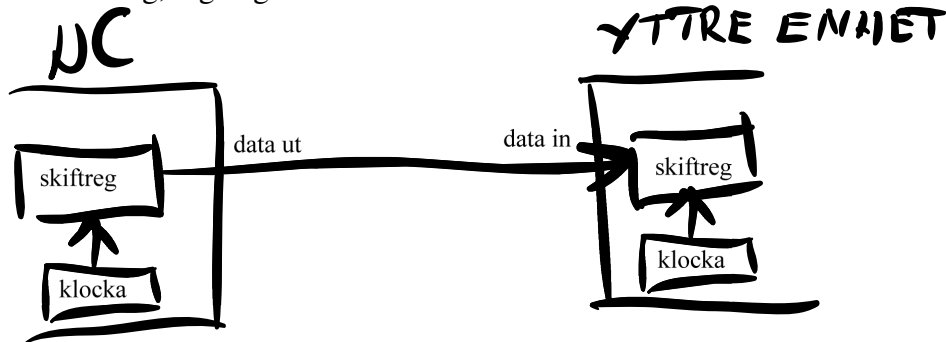
Ø : a utgång, Ø : a ut
1 : a ingång

- Hastigheten 100-400kbit/s
- I²C är bra om man har många enheter som anropas ibland.

UART

Standard för seriell datakommunikation.

RS-232, RS-422, ... är olika elektriska utföranden av samma standard. Här finns enbart en dataledning, ingen gemensam klocka.

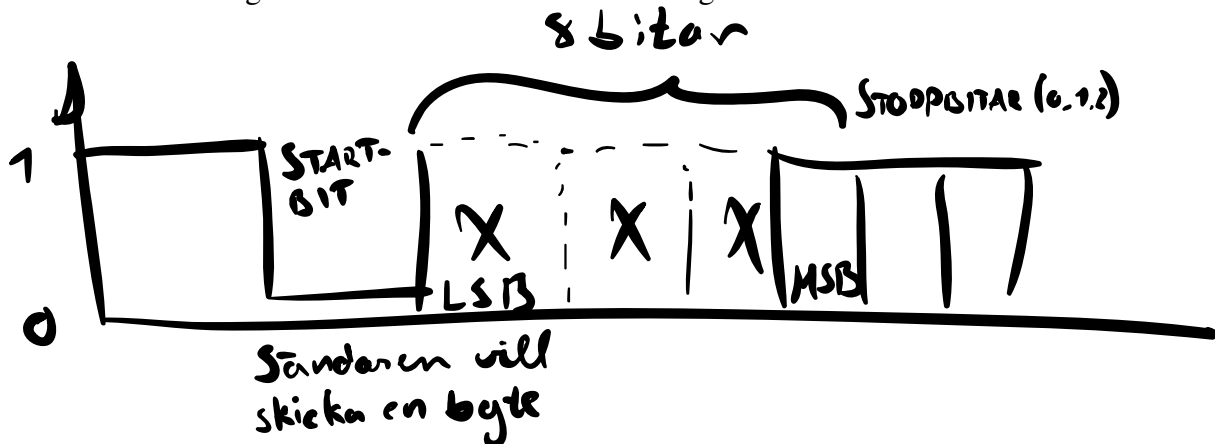


- Innan överföring måste mottagarens klocka ställas till samma frekvens som sändarens.
- Enkelt om känt på förhand.
- Om klockan är okänd kan man ha ett protokoll där känd data skickas flera gånger och mottagaren testat olika frekvenser tills den läser rätt.

Frekvenser är standardiserade, t.ex.

9600 bit/s, 19200 bit/s, 38400 bit/s

- Datalinjen antar 0 eller 1. Nivån är 1 då ingen data skickas.



- Enkel felkontroll kan fås genom paritetsbit. Skickas efter data.
 - Jämn paritet: Antal ettor i data + paritetsbit är jämnt.
 - Udda paritet: Samma fast udda antal ettor.
- Spänningsnivåerna i en RS-232-kabel är inte 0 eller 5 volt:
 - 0: +3V – +18V
 - 1: -3V – -18V

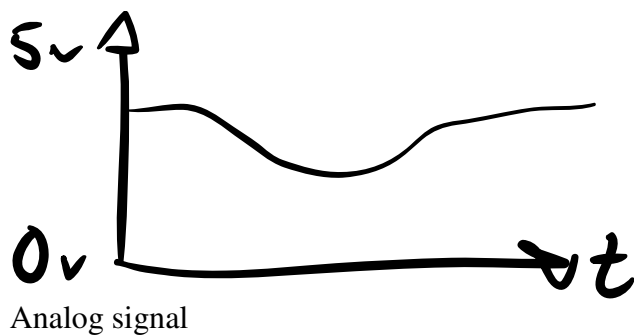
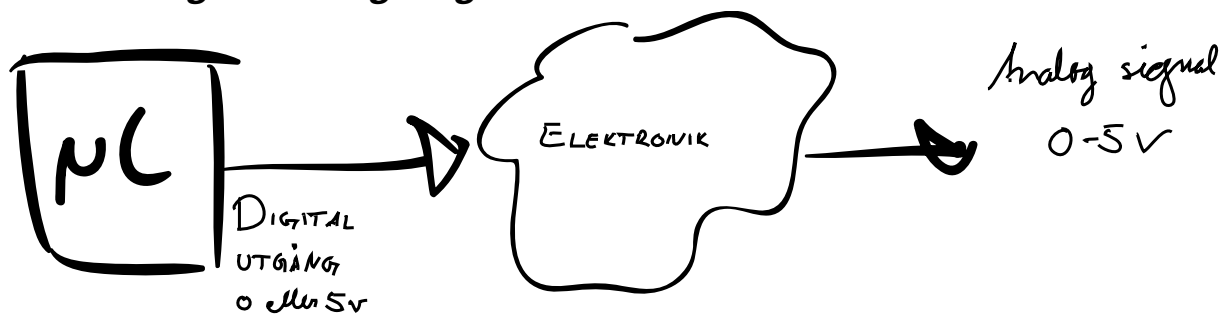
Inbyggda system

Föreläsning 10
060529

Hantering av analoga in- och ut signaler

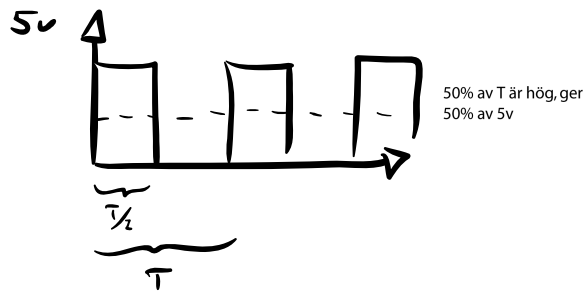
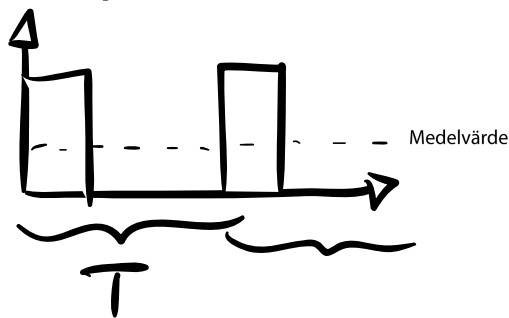
- Enklast: Använd färdig AD/DA-omvandlare
 - mega16: 10 bitars A/D med 8-kanaler
- I många fall klarar man sig utan dessa kretsar, kan göra systemet billigare.

Generering av analoga signaler utan D/A-omvandlare



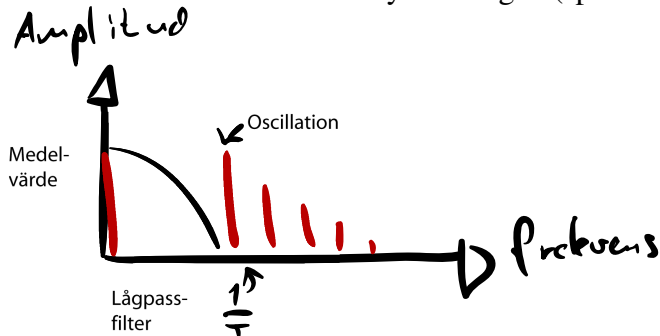
Dessa tider kan betraktas som analoga, även för en digital signal

Exempel.

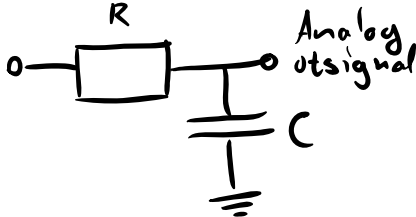


Medelvärdet är en styrbar *analog* utspänning 0-5v.

Studera frekvensinnehållet i fyrkantvågen (spektrum).



Elektronik?



Problem.

Oscillationsfrekvensen hos fyrkantvågen måste vara mycket högre än de frekvenser man vill generera i den analog utsignalen. Annars kräves mer avancerat lågpasfilter.

mega16's pulsbreddsmodulering oscillerar med ~14kHz (beror på upplösning). => Analoga frekvenser upp till 100 Hz är OK!

Många komponenter som vanligtvis styrs med analoga signaler är i sig tröga, t ex lampor, värmeelement eller motorer. De kan inte oscillera lika snabbt som fyrkantvågen.

Mätning av analoga signaler

Elektrisk mätteknik berör mätning av elektriska storheter

- Spänning (volt)
- Ström (ampere)
- Resistans (Ohm)

Icke elektriska storheter mäts genom omvandling till elektriska storheter. Kan vara t ex

- Temperatur
- Tryck

- Kraft
- Läge, hastighet, acceleration

En sådan omvandlare kallas för givare.

Vi vill omvandla de elektriska storheterna till en spänning som vi kan mäta med en A/D-omvandlare.

Exempel.

Antag att vi har en temperaturgivare i form av ett värmekänsligt motstånd (termistor).

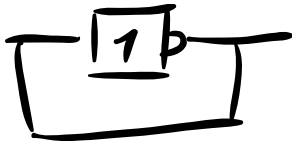
Färdig A/D: Omvandla resistans till spänning med lämplig elektronik.

Utan A/D: Generera en tidsförändring som beror av resistansen och mät denna.

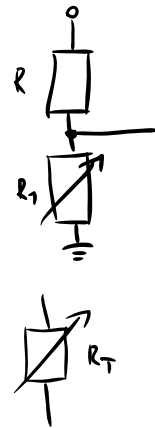
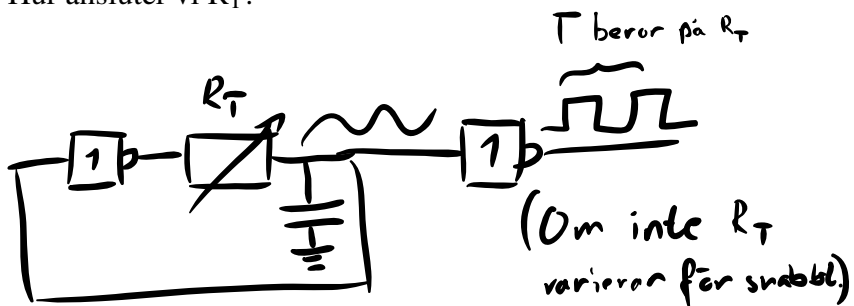
Hur skapa tidsförändring?

1. Oscillator vars frekvens påverkas.
2. Enstaka uppladdning av kondensator.

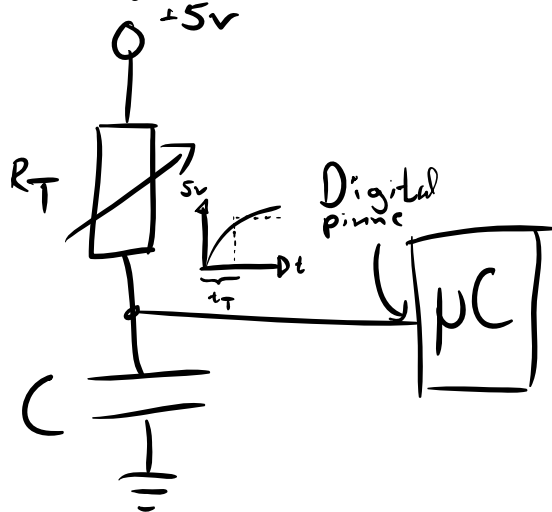
1. Enkel oscillator



Hur ansluter vi R_T ?



2. När börjar vi mäta? Kondensatorn måste vara tom.



Algoritm.

- Låt pinnen vara utgång och lägg ut 0.
- Vänd på pinnen från utgång till ingång och starta en timer.
- Stoppa timern när ingången slår om till 1.

För och nackdelar med ovanstående lösningar:

2) är beroende av vid vilken spänning ingången slår om från 0 till 1. Måste mätas för varje enskild pinne. 1) har inte detta problem. Problemet kan lösas med en extra komponent med väl definierad omslagsnivå, t ex en **Schmitttrigger**.

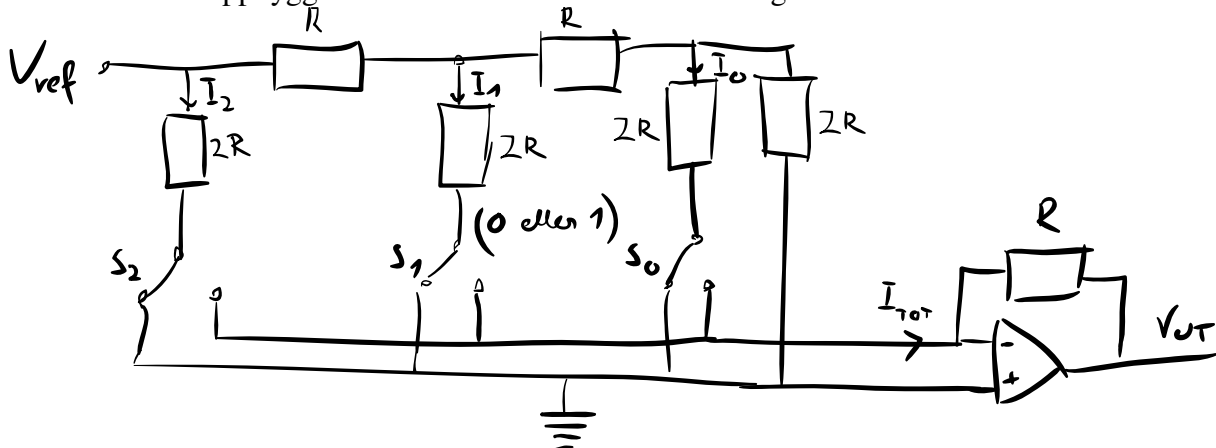
Både 1) och 2) kan ge godtycklig upplösning (antal bitar som vi kan mäta på) eftersom den bestäms av mättiden (väljs av oss) och även timerklockans upplösning.

Inbyggda system

Föreläsning 11
060529

A/D-D/A-omvandlare

De flesta A/D-omvandlare använder sig av något form av D/A-omvandlare. D/A-omvandlare är nästan alltid uppbyggda av ett resistornät kallat R-2R-stege.



$$V_{ut} = \frac{V_{ref}}{2} \cdot S_2 + \frac{V_{ref}}{4} \cdot S_1 + \frac{V_{ref}}{8} \cdot S_0$$

A/D-omvandlare

Vanligaste typerna:

- Successive approximation (SA)
- Single eller dual slope
- Flash

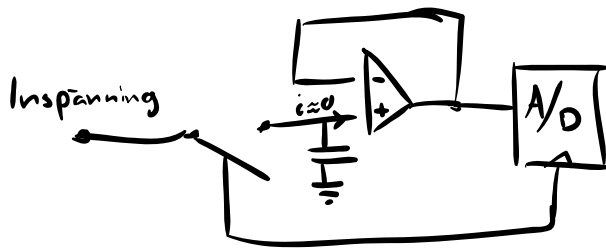
Upplösning anges i antalet bitar. Den minsta spänningsskillnaden som vi kan mäta eller producera är:

$$\frac{V_{ref}}{2^n} \quad n \text{ är antalet bitar}$$

V_{ref} eller referensspänningen är den maximala spänningen vi vill mäta eller producera. Olika typer av A/D-omvandlare är olika snabba. *Flash* är snabbast, *SA* kräver n iterationer för att utföra omvandlingen.

Sample & hold.

Inspänningen som vi vill mäta måste vara konstant under hela omvandlingen för att ge korrekt resultat. (Gäller alla A/D-omvandlare). Man måste alla frysa signalen.



- Switchen är sluten
- "Fryses" genom att A/D-omvandlare öppnar switchen.
- OP:n gör att kondensatorn ej laddas ur.

A/D-omvandlare i mega16

- 10-bits upplösning (SA)
- 8 multiplexade ingångar (PortA)
- A/D-omvandlare behöver referensspänningar för att fungera. Jord AGND, redan ansluten på utvecklingskortet. AREF anger A/D-omvandlarens referensspänning (maximal spänning som mäts). AREF ges av utvecklingskortet som styrs från AVR Studio.

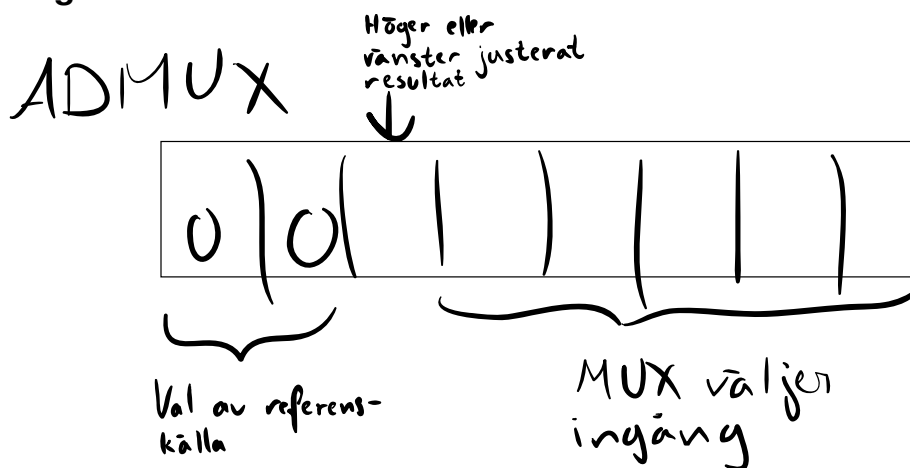
Precision och klockfrekvens

Hög klockfrekvens => lägre precision. Se datablad.

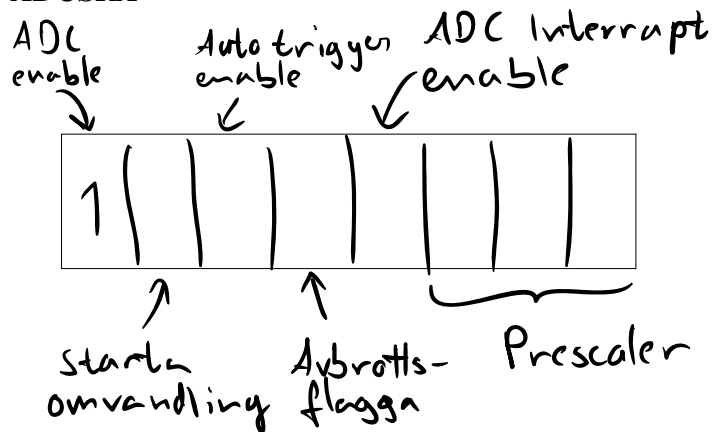
Exempel.

1 MHz klocka till ADC ger 8-bitars noggrannhet. De 8 MSB (most significant bit).

Register i A/D-omvandlaren



ADCSRA



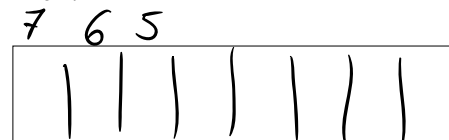
ADC Data Register

ADCL och ADCH (L = low byte, H = high byte)

SFIOR

bit 5-7 anger val av triggerkälla till A/D-omvandlingar.

Tex.

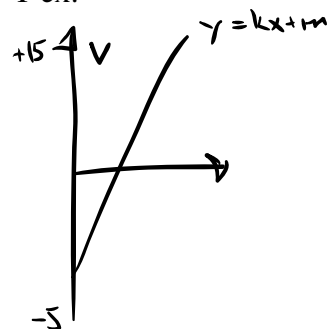


0	0	0	Free running (Auto reboot)
0	1	0	External interrupt 0
1	0	0	Timer 0 overflow

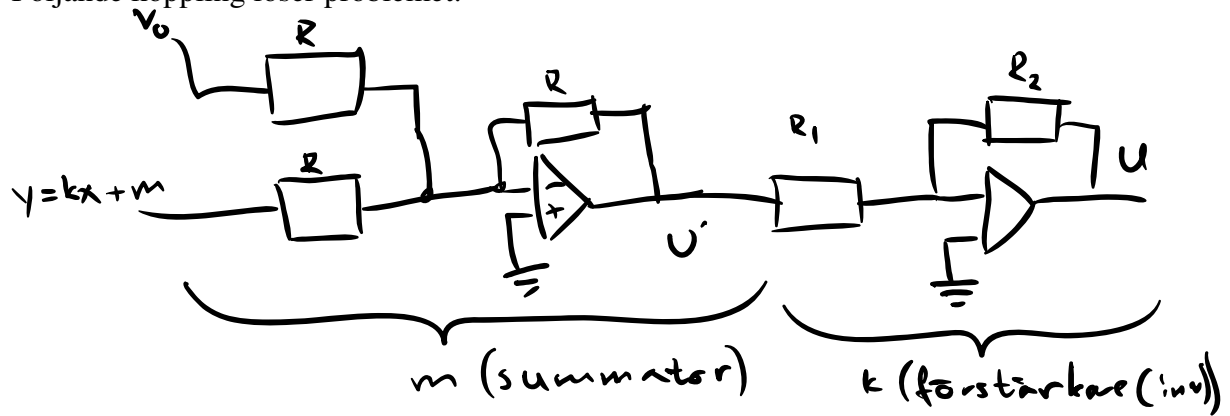
Linjäritet.

Antag en mätkurva som är linjär men ligger i fel intervall.

Tex.



Följande koppling löser problemet.



$$U' = -(U_0 + y) = -(U_0 + kx + m)$$

Med U_0 kan vi flytta kurvan

$$U_0 = -m \Rightarrow \text{kurvan startar i origo.}$$

$$k : U = -\frac{R_2}{R_1} U' = -\frac{R_2}{R_1} \cdot (-1) kx = \frac{R_2}{R_1} kx \quad (\text{m-värdet är } 0)$$

$$\frac{R_2}{R_1} k \text{ är den nya lutningen.}$$

$$\frac{R_2}{R_1} = \frac{1}{4} \text{ ger } 5V \text{ då } x = 1$$